

Контроль использования индексов

Как уже говорилось, проектирование эффективных индексов имеет первостепенную важность для достижения высокой производительности БД и приложений.

Понять, насколько эффективен созданный индекс, и будет ли он использоваться в определенных запросах, поможет команда EXPLAIN — специальный инструмент PostgreSQL, работа которого описана в этом документе.

Выполняя любой полученный запрос, планировщик PostgreSQL разрабатывает для него [план запроса](#). Для получения одного результата могут использоваться разные способы; планы запросов могут существенно отличаться и по затратам ресурсов, и по времени выполнения. Поэтому PostgreSQL содержит специальный оптимизатор (CBO: Cost Based Optimizer), предназначенный для того, чтобы из всех сгенерированных планов определить оптимальный, наиболее "дешевый", план запроса.

Команда EXPLAIN выводит план выполнения, генерируемый планировщиком для заданного запроса. С помощью этой команды можно узнать, как отдельный запрос использует индексы.

Работа с планами выполнения возможна в

- [pgAdmin](#) — среда администрирования и разработки для PostgreSQL;
- [explain.sbis.ru](#) — сервис СБИС для анализа работы серверов PostgreSQL.

В этой статье приведена основная информация по EXPLAIN, для удобства изучения дополнительно описаны [примеры работы](#).

Для подробного изучения настоятельно рекомендуем обратиться к материалам, ссылки на которые приведены в конце этой [статьи](#).

EXPLAIN

Синтаксис команды:

```
1 EXPLAIN [ ( параметр [, ...] ) ] оператор
2 EXPLAIN [ ANALYZE ] [ VERBOSE ] оператор
3
4 Здесь допускается параметр:
5
6     ANALYZE [ boolean ]
7     VERBOSE [ boolean ]
8     COSTS [ boolean ]
9     BUFFERS [ boolean ]
10    TIMING [ boolean ]
11    FORMAT { TEXT | XML | JSON | YAML }
```

Описание параметров:

- **ANALYZE**



Выполнить команду и вывести фактическое время выполнения и другую статистику. По умолчанию FALSE. С параметром ANALYZE оператор будет выполнен на самом деле, а не только запланирован.

При этом в вывод добавляются фактические сведения о времени выполнения, включая общее время, затраченное на каждый узел плана (в миллисекундах) и общее число строк, выданных в результате. Это помогает понять, насколько близки к реальности предварительные оценки планировщика.

Хотя EXPLAIN отбрасывает результат, который вернул бы SELECT, в остальном все действия выполняются как обычно. Если нужно выполнить EXPLAIN ANALYZE с командой INSERT, UPDATE, DELETE, CREATE TABLE AS или EXECUTE, не допуская изменения данных этой командой, воспользуйтесь таким приёмом:

```
1 BEGIN;
2 EXPLAIN ANALYZE ...;
3 ROLLBACK;
```

- **VERBOSE**

Вывести дополнительную информацию о плане запроса. По умолчанию FALSE.

- **COSTS**

Вывести рассчитанную стоимость запуска и общую стоимость каждого узла плана, а также рассчитанное число строк и ширину каждой строки. По умолчанию TRUE.

- **BUFFERS**

Включить информацию об использовании буфера. Параметр действует только в режиме ANALYZE. По умолчанию FALSE.

- **TIMING**

Включить в вывод фактическое время запуска и время, затраченное на каждый узел. Постоянное чтение системных часов может значительно замедлить запрос, так что если достаточно знать фактическое число строк, имеет смысл сделать этот параметр равным FALSE. Параметр действует только в режиме ANALYZE. По умолчанию TRUE.

- **FORMAT**

Установить один из следующих форматов вывода: TEXT, XML, JSON или YAML. Последние три формата содержат ту же информацию, что и текстовый, но больше подходят для программного разбора. По умолчанию TEXT.

- **boolean**

Включает или отключает заданный параметр. Для включения параметра можно написать TRUE, ON или 1, а для отключения — FALSE, OFF или 0. Значение boolean можно опустить, в этом случае подразумевается TRUE.

- **оператор**

Любой оператор SELECT, INSERT, UPDATE, DELETE, VALUES, EXECUTE, DECLARE, CREATE TABLE AS и CREATE MATERIALIZED VIEW AS, план выполнения которого вас интересует.

Структура плана запроса

Посмотреть план выполнения можно, набрав EXPLAIN перед запросом, выполнение которого нужно проанализировать:

Редактор SQL Графический конструктор запросов

Предыдущие запросы

```
explain select * from itshop p1, shop p2
where p1.man =50 and p1.man = p2.man;
```

Панель вывода

Вывод данных Построить план выполнения Сообщения История

	QUERY PLAN
	text
1	Nested Loop (cost=18.90..36875.30 rows=639639 width=250)
2	-> Index Scan using itshop tgr nom man idx on itshop p1 (cost=0.42..26225.42 rows=819 width=126)
3	Index Cond: (man = 50)
4	-> Materialize (cost=18.48..2656.34 rows=781 width=124)
5	-> Bitmap Heap Scan on shop p2 (cost=18.48..2652.44 rows=781 width=124)
6	Recheck Cond: (man = 50)
7	-> Bitmap Index Scan on shop man idx (cost=0.00..18.28 rows=781 width=0)
8	Index Cond: (man = 50)

Структура плана запроса представляет собой дерево в текстовом представлении:

QUERY PLAN

text

```
Nested Loop (cost=1082.55..3855.70 rows=783 width=169)
  -> Seq Scan on manufacturer p2 (cost=0.00..2.00 rows=1 width=43)
      Filter: (kod = 6)
  -> Bitmap Heap Scan on itshop p1 (cost=1095.48..3737.80 rows=783 width=126)
      Recheck Cond: ((man = 6) AND (nom < 11000000))
      -> BitmapAnd (cost=1095.48..1095.48 rows=783 width=0)
          -> Bitmap Index Scan on itshop_man_idx (cost=0.00..498.14 rows=25295 width=0)
              Index Cond: (man = 6)
          -> Bitmap Index Scan on itshop_nom_idx (cost=0.00..596.70 rows=29903 width=0)
              Index Cond: (nom < 11000000)
```

В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана — **cost**; именно это значение планировщик старается минимизировать.

Узлы на нижнем уровне дерева — это узлы сканирования, которые возвращают необработанные данные таблицы. Разным типам доступа к таблице соответствуют разные узлы:

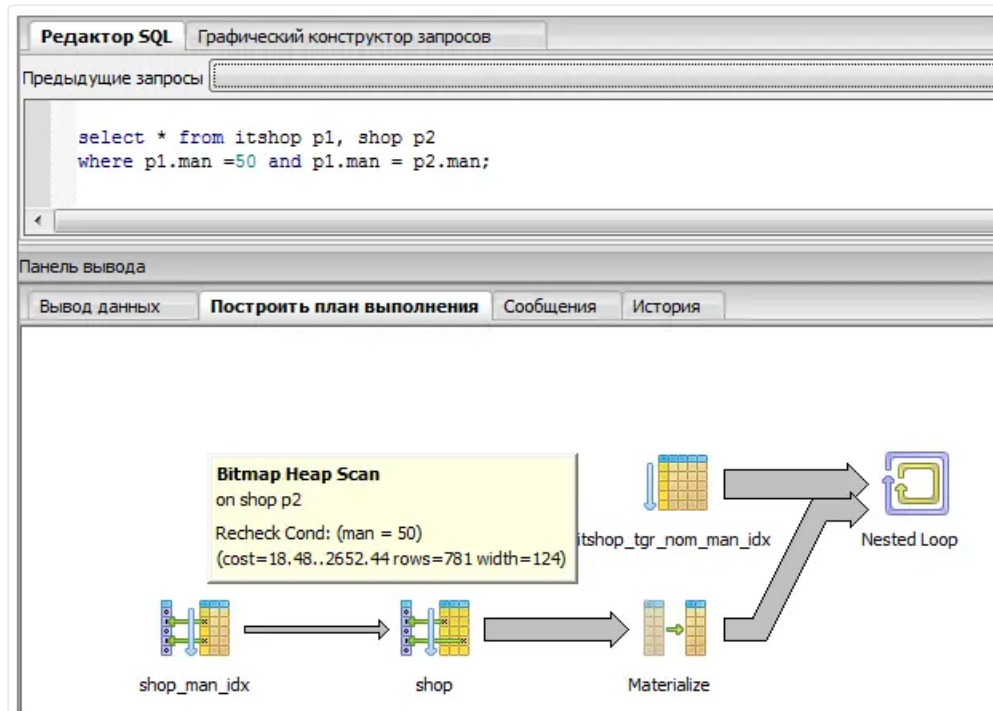
- последовательное сканирование,
- сканирование индекса,
- сканирование битовой карты ([bitmap](#)).

Если запрос требует объединения, агрегатных вычислений, сортировки или других операций с исходными строками, над узлами сканирования появляются узлы, обозначающие эти операции.

Так как обычно операции могут выполняться разными способами, на этом уровне тоже могут быть узлы разных типов.

В выводе команды EXPLAIN для каждого узла в дереве плана отводится одна строка, где показывается базовый тип узла плюс оценка стоимости выполнения данного узла, которую сделал для него планировщик. Если для узла выводятся дополнительные свойства, в вывод могут добавляться дополнительные строки, с отступом от основной информации узла.

Структуру плана также можно вывести в графическом режиме, в виде схемы:



Несколько советов по работе с EXPLAIN

Для того чтобы работа с EXPLAIN, была продуктивной:

- Используйте в экспериментах реальные данные. Для анализа в тестовой БД лучше создать таблицу с реальными (приближенными к реальным) данными.

Анализируя работу системы с тестовыми данными, можно понять, какие индексы нужны для тестовых данных, но не более того. Если вы наполняете базу данных очень близкими, или наоборот, случайными значениями, либо добавляете строки в отсортированном порядке, вы получите совсем не ту статистику распределения, что дадут реальные данные.

- Искажают картину и очень маленькие наборы тестовых данных. Если для извлечения 100 строк из 100000 может быть применён индекс, для выбора 1 из 100 он вряд ли потребуется.
- Всегда начинайте исследование с [ANALYZE](#). Эта команда собирает статистические данные о распределении значений в таблице.

Данные нужны для оценки числа строк, возвращаемых запросами. Число строк требуется планировщику, чтобы оценить реальные затраты для всевозможных планов выполнения запроса. Не имея реальной статистики, планировщик примет некоторые значения по умолчанию, которые почти наверняка не будут соответствовать действительности.

Поэтому понять, как индекс используется приложением, без предварительного запуска ANALYZE, практически невозможно.

- Когда индексы не используются, ради тестирования может быть полезно подключить их принудительно, настраивая [параметры конфигурации методов планировщика](#).
- Один из способов посмотреть различные планы — **принудить планировщик** не считать выбранную им стратегию самой выгодной, настраивая [параметры конфигурации методов планировщика](#), про которые говорилось пунктом выше. Это полезный, хотя и грубый инструмент. Описание применения принуждения планировщика приведено в пункте 2.4 раздела [WHERE](#).
- Если система использует индекс только под принуждением, возможны две причины:
 - применять индекс неэффективно,
 - оценка стоимости применения индекса не соответствует действительности.

В этом случае следует замерить время выполнения запроса с индексами и без них. В анализе этой ситуации может быть полезна команда EXPLAIN ANALYZE.
- Если оценка стоимости неверна, возможны две причины:
 - Общая стоимость вычисляется как произведение цены каждого узла плана для одной строки и оценки избирательности узла плана. В случае крайней необходимости цены узлов можно изменить, изменив [значение параметров](#).
 - Оценка избирательности может быть неточной из-за некачественной статистики. Улучшить её можно, настроив параметры сбора статистики с помощью [ALTER TABLE](#).

Примеры работы с EXPLAIN

В БД PostgreSQL создадим таблицу товаров интернет-магазина itshop с полями:

- nom — номенклатурный номер
- name — наименование
- kol — количество
- price — цена
- man — производитель

```
CREATE TABLE itshop (nom integer, name text, kol integer, price
numeric(32,2), man integer);
```

и импортируем в таблицу данные из прайс-листа магазина (1 019 232 строки);
 файл с данными для импорта можно взять [здесь](#).

1. ANALYZE. Команда и параметр

1.1

Запустим простой запрос:

```
explain select * from itshop;
```

QUERY PLAN
text

Seq Scan on itshop (cost=0.00..29509.32 rows=1019232 width=120)

EXPLAIN выдаст сообщение, что используется Seq Scan — последовательное, блок за блоком, чтение данных таблицы itshop. Расшифруем параметры, записанные в скобках:

- **cost** — приблизительная стоимость:
 - 0.00 — приблизительная стоимость запуска — вычисляется за время, которое проходит до начала вывода данных;
 - 29509.32 — приблизительная общая стоимость — вычисляется с предположением, что узел плана выполняется до конца, то есть возвращает все доступные строки;стоимость измеряется в произвольных единицах, определяемых [параметрами планировщика](#); важно понимать, что стоимость узла верхнего уровня включает стоимость всех его потомков; стоимость отражает только те факторы, которые учитывает планировщик — в определении стоимости, например, не учитывается время передачи результатов запроса клиенту;
- **rows** — ожидаемое приблизительное число строк, которое должен вывести узел плана — не обработанных или просканированных, а именно выданных; предполагается, что узел выполняется до конца; в идеале, на верхнем уровне это значение будет приблизительно равно числу строк, которое фактически возвращает (удаляет/изменяет) запрос;
- **width** — ожидаемый средний размер строк, выводимых этим узлом плана (в байтах).

Добавим в таблицу еще 8 строк и повторим наш запрос. Значение rows не изменилось, статистика по таблице старая.

1.2

Для обновления статистики вызываем команду ANALYZE и повторяем запрос:

```
1 analyze itshop;  
2 explain select * from itshop;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..29902.40 rows=1019240 width=124)

Как видим, количество строк изменилось; затраты на получение всех строк тоже изменились. При выполнении ANALYZE:

- считывается определённое количество строк таблицы, выбранных случайным образом;
- собирается статистика значений по каждой из колонок таблицы.

Сколько строк будет считывать ANALYZE — зависит от параметра [default_statistics_target](#).

1.3

Данные, полученные выше — только ожидания планировщика. Сверим их с результатами реального запроса, то есть используем параметр ANALYZE:

```
explain (analyze) select * from itshop;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..29902.40 rows=1019240 width=124) (actual time=0.021..105.271 rows=1019240 loops=1)
Planning time: 0.166 ms
Execution time: 118.505 ms

Как видим, в выводе команды добавились параметры:

- actual time — реальное время в миллисекундах, затраченное для получения первой строки и всех строк соответственно.
- loops — количество проходов узла плана (циклов).

2. WHERE

2.1

Добавим в запрос условие — выберем из нашей таблицы информацию по всем товарам, исключив товарные группы от 10 до 19:

```
explain select * from itshop where nom > 19999999;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..32450.50 rows=978473 width=124)
Filter: (nom > 19999999)

Индексов у таблицы нет. При выполнении запроса последовательно считывается каждая запись таблицы (Seq Scan). Каждая запись сравнивается с условием. Если условие выполняется, запись вводится в результат. Иначе — отбрасывается. Filter означает именно такое поведение. Значение cost, что логично, увеличилось.

2.2

Добавим в таблицу индекс и повторим запрос:

```
1 create index on itshop(nom);
2 explain select * from itshop where nom > 19999999;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..32450.50 rows=978473 width=124)
Filter: (nom > 19999999)

Как видно, индекс не был использован, результаты не изменились.

2.3

Сверим их с результатами реального запроса:

```
explain (analyze) select * from itshop where nom > 19999999;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..32450.50 rows=978473 width=124) (actual time=0.033..145.108 rows=981943 loops=1)
Filter: (nom > 19999999)
Rows Removed by Filter: 37297
Planning time: 0.661 ms
Execution time: 157.926 ms

Из более чем миллиона строк отфильтровано только 37 297. То есть, для получения результата обработано более 96% данных.

2.4

Если **заставить** использовать индекс:

```
1 set enable_seqscan to off;
2 explain (analyze) select * from itshop where nom > 19999999;
```

QUERY PLAN
text
Bitmap Heap Scan on itshop (cost=18327.59..50268.50 rows=978473 width=124) (actual time=63.564..200.346 rows=981943 loops=1)
Recheck Cond: (nom > 19999999)
Heap Blocks: exact=18851
-> Bitmap Index Scan on itshop nom idx (cost=0.00..18082.97 rows=978473 width=0) (actual time=60.342..60.342 rows=981943 loops=1)
Index Cond: (nom > 19999999)
Planning time: 0.079 ms
Execution time: 213.230 ms

Мы увидим, что значения cost и actual time увеличились. При выборке практически всей таблицы планировщик не будет использовать индекс — использование индекса только увеличивает затратность операции и время выполнения запроса.

2.5

Отменим запрет на использование Seq Scan и изменим запрос. Выберем информацию только по товарным группам от 10 до 19:

```
1 set enable_seqscan to on;
2 explain select * from itshop where nom < 20000000;
```

QUERY PLAN
text
Bitmap Heap Scan on itshop (cost=764.37..20983.96 rows=40767 width=124)
Recheck Cond: (nom < 20000000)
-> Bitmap Index Scan on itshop nom idx (cost=0.00..754.18 rows=40767 width=0)
Index Cond: (nom < 20000000)

Теперь индекс работает. В данном случае используется план из двух этапов:

1. Дочерний узел плана просматривает индекс (Bitmap Index Scan) и находит в нем адреса строк, соответствующих условию (Index cond).
2. Верхний узел выбирает эти строки из таблицы (Bitmap Heap Scan).

2.6

Добавим в запрос еще одно условие:

```
explain select * from itshop where nom < 20000000 and name like
'Монитор%';
```

QUERY PLAN
text
Bitmap Heap Scan on itshop (cost=754.58..21076.08 rows=1606 width=124)
Recheck Cond: (nom < 20000000)
Filter: (name ~~ 'Монитор% '::text)
-> Bitmap Index Scan on itshop nom idx (cost=0.00..754.18 rows=40767 width=0)
Index Cond: (nom < 20000000)

Добавленное условие уменьшает оценку количества результирующих строк, но не стоимость (она даже возросла), так как просматриваться будет тот же набор строк, что и раньше. Видно также, что условие на пате не добавилось в качестве условия индекса (так как индекс определен по полю nom), оно применяется как фильтр к строкам, полученным по индексу. Поэтому и стоимость возросла.

2.7

Если выбирать не всю строку, а только поле, по которому создан индекс:

```
explain select nom from itshop where nom < 20000000;
```

QUERY PLAN
text
Index Only Scan using itshop nom idx on itshop (cost=0.42..1156.19 rows=38615 width=4)
Index Cond: (nom < 20000000)

Планировщик выберет Index Only Scan. Запрос выполняется быстрее, потому что не нужно брать данные всей строки таблицы — сканирование только по индексу (width=4).

3. AND и OR

3.1

Если в таблице есть отдельные индексы по разным колонкам, фигурирующим в WHERE, планировщик может выбрать сочетание этих индексов (с AND и OR):

```
1 create index on itshop(man);
2 explain select nom from itshop where nom < 20000000 and man = 6;
```

QUERY PLAN
text
Bitmap Heap Scan on itshop (cost=1251.67..4711.00 rows=1068 width=4)
Recheck Cond: ((man = 6) AND (nom < 20000000))
-> BitmapAnd (cost=1251.67..1251.67 rows=1068 width=0)
-> Bitmap Index Scan on itshop man idx (cost=0.00..496.70 rows=26704 width=0)
Index Cond: (man = 6)
-> Bitmap Index Scan on itshop nom idx (cost=0.00..754.18 rows=40767 width=0)
Index Cond: (nom < 20000000)

Но для этого потребуется обойти оба индекса, так что это не обязательно будет выгоднее, чем просто просмотреть один индекс, а второе условие обработать как фильтр.

3.2

Если изменить диапазон, то можно это увидеть:

```
explain select nom from itshop where nom < 20000000 and man > 6;
```

QUERY PLAN
text
Bitmap Heap Scan on itshop (cost=775.23..22298.94 rows=20752 width=4)
Recheck Cond: (nom < 20000000)
Filter: (man > 6)
-> Bitmap Index Scan on itshop nom idx (cost=0.00..770.04 rows=38615 width=0)
Index Cond: (nom < 20000000)

3.3

Создадим [составной индекс](#) и посмотрим, как выполнится запрос:

```
explain select * from itshop where tgr = 35 and nom between 35000500
and 350500000 and man = 10;
```

QUERY PLAN
text
Index Scan using itshop tgr nom man idx on itshop (cost=0.42..25868.79 rows=1927 width=126)
Index Cond: ((tgr = 35) AND (nom >= 35000500) AND (nom <= 350500000) AND (man = 10))

Планировщик использует просмотр по индексу (Index Scan).

3.4

Поменяем условие запроса, расширив диапазон выборки:

```
explain select * from itshop where tgr = 35 and nom between 35000500
and 350500000 and man > 10;
```

QUERY PLAN
text
Seq Scan on itshop (cost=0.00..60389.52 rows=456863 width=126)
Filter: ((nom >= 35000500) AND (nom <= 350500000) AND (man > 10) AND (tgr = 35))

Как видно, индекс не используется планировщиком.

В условиях выборки большего объема данных планировщик посчитал последовательную выборку из таблицы дешевле использования индекса.

Материалы, рекомендуемые к изучению

1. Боровиков К. ["PostgreSQL для разработчика. Оптимизация запросов"](#).

Кратко, наглядно и подробно описаны отображаемые параметры и методы сканирования. Для изучения обязателен.

2. ["EXPLAIN"](#).

Документация официального сайта PostgreSQL.

3. ["EXPLAIN"](#).

Документация на сайте российских разработчиков PostgreSQL.

4. Хабрахабр. ["Объясняя необъяснимое"](#).

5. [Сервис explain.sbis.ru](#).

Дополнительно рекомендуется к просмотру видеодоклад Бартунова О. Короткова А. [Вся правда об индексах в PostgreSQL](#), в котором российские разработчики рассказывают про индексы, алгоритм выбора нужного индекса и методы сканирования.